USED MEDIAS LLC
EMBEDDED SYSTEMS DIVISION

Xtensa ABI Selection

UM-NATOS-003

DOCUMENT	REVISION	STATUS	CLASSIFICATION
UM-NATOS-003	1.0 · 2026-08-14	current — verified by codegen inspection	Internal

1. Abstract

The Xtensa LX6 supports two calling conventions: the windowed ABI and the call0 ABI. nat-os is built entirely with **call0**. This report records the evidence behind that decision, what it costs, and why it is the single change that makes a from-scratch scheduler tractable on this architecture.

This decision is load-bearing. It cannot be revisited cheaply once drivers and the VM exist, because ABIs cannot be mixed within a link.

2. Background — the windowed ABI

The Xtensa architecture optionally implements a **register window**: 64 physical address registers of which 16 are visible, with the visible window rotating on call and return. `ENTRY` rotates the window and allocates a stack frame; `RETW` reverses it.

When the window wraps, the hardware raises a **window overflow exception**, and the handler must spill registers to the stack. Returning past a spilled frame raises **window underflow**, and the handler must reload them.

Consequences for an operating system:

- A context switch cannot simply save 16 registers. Live windows belonging to the outgoing task exist in the physical register file and must be spilled first, or restored state will be silently wrong.
- The spill mechanism must work during the switch itself, which constrains what the switch code may touch.
- Failures do not manifest at the switch. They manifest several returns later, in a function that did nothing wrong, with a corrupted frame.

This is the single hardest part of writing an Xtensa kernel and the point at which such projects most often stall.

3. The call0 alternative

The call0 ABI does not use windows. It behaves as a conventional RISC calling convention:

Register	Role under call0
`a0`	Return address
`a1`	Stack pointer
`a2` – `a7`	Arguments / return values
`a12` – `a15`	Callee-saved

No `ENTRY`, no `RETW`, no window exceptions. **A context switch becomes a conventional register save and restore.**

4. Verification

The ABI was not assumed to be available — it was tested against the installed toolchain (`xtensa-esp32-elf-gcc`, crosstool-NG esp-14.2.0_20241119) before any project code was written.

Test input:

```
int add(int a, int b) { return a + b; }
int chain(int x)      { return add(x, add(x, x)); }
```

Compiled at `-O2` under each ABI. Relevant emitted instructions:

`-mabi=windowed` (default)	`-mabi=call0`
`entry sp, 32`	*(none)*
`retw.n`	`ret.n`

The windowed build allocates a window frame on entry and returns with `RETW`. The call0 build emits neither — a plain `ret.n`, with no window management at all.

Result: call0 is supported by this toolchain for this target. Confirmed again in the linked kernel; the disassembly of `_start` shows the `.bss` clear loop using `bgeu` / `s32i.n` / `addi.n` with no `entry` or `retw` present.

5. Costs and constraints

5.1 ROM routines are unusable

The ESP32 ROM exports functions — memory routines, SPI helpers, cryptography — all compiled for the **windowed** ABI. Calling them from call0 code will corrupt state.

Impact: low for this project. A from-scratch OS writes its own drivers by definition. Where a ROM routine is genuinely wanted, an assembly wrapper that switches conventions is possible, but none is currently needed.

Concrete instance: `kernel/uart.c` writes UART registers directly instead of calling the ROM's output routines, specifically for this reason.

5.2 All code must use one ABI

Every object in the link must agree. Any third-party library compiled as windowed cannot be linked in. This closes off casual reuse of ESP-IDF components — consistent with the project's scope, but worth stating explicitly because it will resurface the first time a tempting library appears.

5.3 Slightly larger code

Without windows, callee-saved registers are spilled to the stack conventionally. Code is typically marginally larger than the windowed equivalent. At 1,124 bytes of text this is not currently measurable and is not expected to matter.

6. Consequence for the scheduler (M2)

With call0, a task switch is:

1. Push `a0`, `a12` – `a15` and any live caller-saved registers onto the outgoing task's stack.
2. Store the resulting stack pointer in the outgoing task's control block.
3. Load the incoming task's stack pointer.
4. Pop the registers.
5. `ret` — which returns into the incoming task.

No spilling, no window state, no exception interaction. This is comparable in difficulty to a Cortex-M context switch, and is the reason M2 is expected to be a manageable milestone rather than the project's wall.

The VM changes the picture further. Because application scheduling happens at bytecode-instruction boundaries inside the interpreter (UM-NATOS-001 §4.2), native context switching is required only for kernel and driver tasks. The number of native tasks is expected to stay small.

7. Build flags

Set in `build.ps1` for both compile and link:

```
-mabi=call0              select the ABI
-mtext-section-literals  place literal pools in .text so they land in IRAM
-mlongcalls              allow calls beyond the short-call displacement range
```

`-mabi=call0` must appear on the **link** command as well as the compile commands; the driver uses it to select compatible internal libraries.

8. Residual risk

- **Exception and interrupt handlers.** The interrupt vectors (M1) are entered by hardware, not by a call. Their entry sequence must save state explicitly and correctly; call0 simplifies but does not eliminate this work.
- **Toolchain internals.** `libgcc` routines pulled in for operations such as 64-bit division must also be call0. The current build links `-nostdlib` and uses none, but that will change and should be re-checked at that point.

9. References

- UM-NATOS-001 §4 — application isolation model
- UM-NATOS-005 — full compiler and linker flag list
- Xtensa Instruction Set Architecture Reference Manual — window exception behaviour